

Programación II

Desarrollo en Java

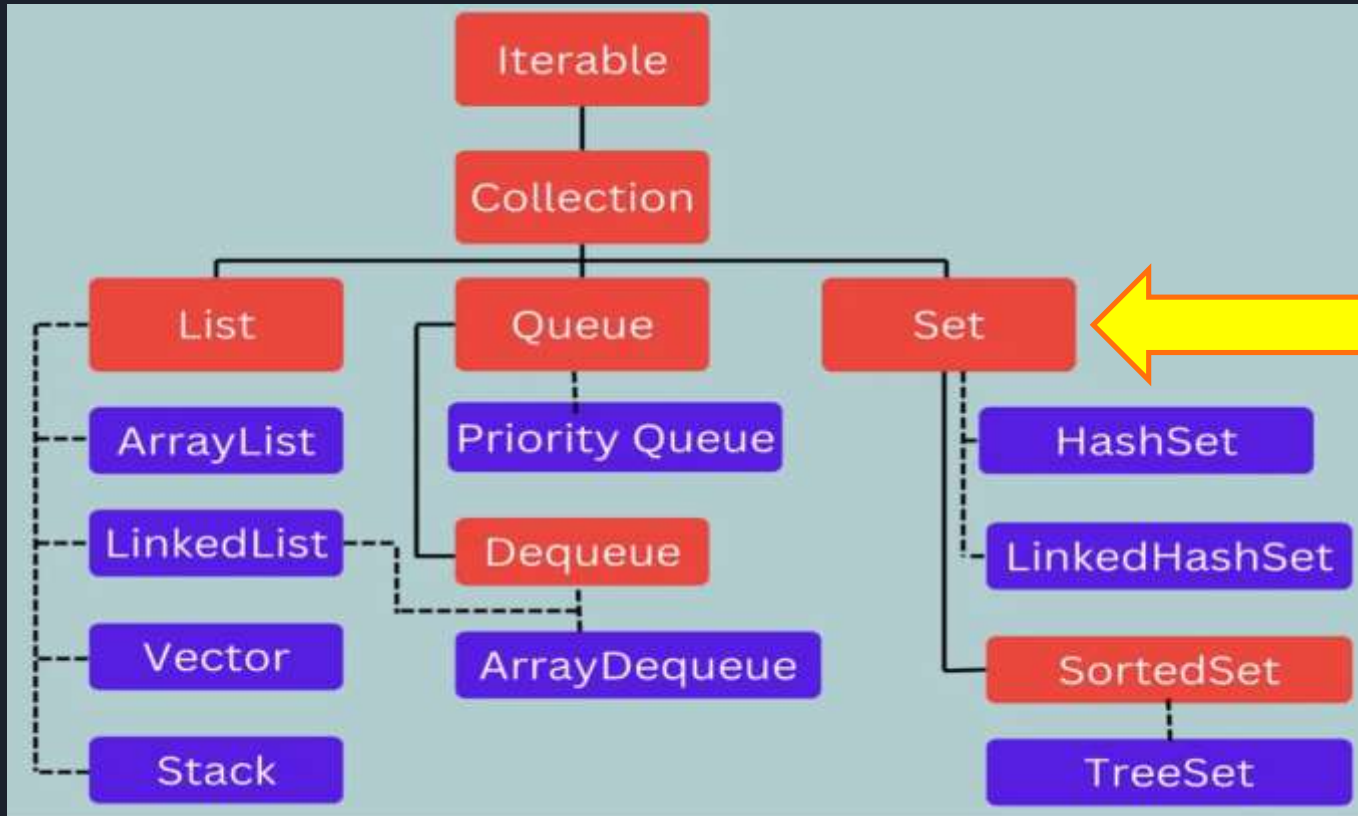
Clase N° 13:

Colecciones de tipo Set.

Docente Carolina Archuby.



INTERFAZ SET (conjuntos)



Interfaz Set. Características principales

- * La interfaz Set se utiliza para representar una colección de elementos:
- => **únicos:** La interfaz Set hereda los métodos de Collection pero agrega sus propias restricciones para prohibir el duplicado de elementos.
- => **no ordenados:** No se puede garantizar el orden en el que se devolverán los elementos.
- => **Con acceso mediante iteración:** No se puede acceder a los elementos por índice, como en List, se accede mediante un iterador.
- * La interfaz Set es implementada en las clases: **HashSet, TreeSet Y LinkedHashset**

Interface Set: El método add y los objetos repetidos:

=> si tenemos un objeto que tiene las mismas características (equals) y el mismo hashcode que algún objeto que ya se encuentra en el Set , no se lanza ninguna excepción, pero **el elemento NO se agrega a la colección**

=> Como el método **add** de la Interface Collection retorna true o false si agregó o no, en este caso nos sirve analizarlo: **si el elemento a añadir no está en el conjunto y ha sido añadido retornará true, o false si el elemento ya se encontraba dentro del conjunto.**

=> Un conjunto podrá contener **a lo sumo un elemento null.**

Interface Set: El método add y los objetos repetidos:

Para que Set funcione correctamente, es crucial que los objetos que se añaden implementen de manera adecuada los métodos `equals` y `hashCode`

- `equals(Object obj)`: Determina si dos objetos son iguales en contenido. Es usado por implementaciones de Set para asegurar que no se añadan duplicados.
- `hashCode()`: Proporciona el código hash de un objeto, que es usado por las implementaciones basadas en hash de Set (como HashSet) para almacenar objetos de manera que puedan ser recuperados rápidamente.

Si dos objetos son considerados iguales según el método `equals`, entonces deben tener el mismo valor de código hash

Interface Set: Cómo recorrer la colección:

=> PARA RECORRER UN SET SE USA UN ITERADOR.

=> `Iterator iterator()`: método que retorna un iterador (una especie de cursor o puntero lógico), que se ubica en una posición antes del primero (por eso se usa `next`)

```
Iterator it = hs.iterator();  
  
while(it.hasNext())  
{  
    System.out.println(it.next());  
}
```

Interface Set: Métodos.

- => Set **hereda todos los métodos de la interfaz Collection** (add, remove, contains, size, etc).
- => No introduce nuevos métodos específicos.
- => Su principal diferencia radica en cómo maneja la adición de elementos y la garantía de unicidad, agregando sus propias restricciones para prohibir el duplicado de elementos.

Interface Set: Métodos para comparar colecciones:

Cuando se comparan dos conjuntos para igualdad, se consideran iguales si contienen exactamente los mismos elementos, independientemente del orden.

Esto es diferente de las listas, donde el orden de los elementos también importa.

La igualdad entre dos Set se comprueba utilizando los métodos equals de los elementos contenidos, verificando que cada conjunto contenga todos los elementos del otro sin considerar el orden.

- `boolean addAll(Collection<? extends E> var1);`

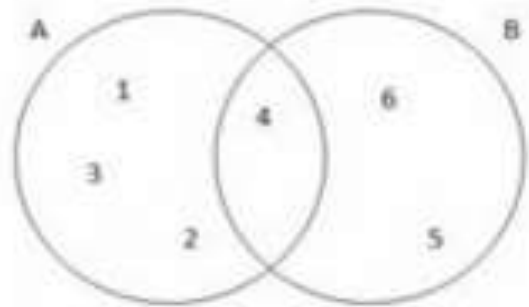
$A \cup B = \{1,2,3,4,5,6\}$

- `boolean retainAll(Collection<?> var1);`

$A \cap B = \{4\}$

- `boolean removeAll(Collection<?> var1);`

$A - B = \{1,2,3\}$



HASHSET

=> Los objetos se almacenan en una **TABLA DE DISPERSIÓN (HASH)**: Almacena los elementos utilizando el código hashCode para determinar en qué parte de la tabla se almacenará el elemento.

=> La clase HashSet **delega casi todas sus funcionalidades a un mapa interno** (HashMap)

=> **LOS ELEMENTOS NO NECESARIAMENTE SE MOSTRARÁN EN EL ORDEN EN QUE SE INSERTARON.**

HashSet: “Costo” de las operaciones

=> La tabla hash hace que sea mas eficiente que las clases que implementan la interfaz List en las operaciones de inserción, borrado y búsqueda, ya que proporciona tiempos constantes (siempre y cuando la función hash disperse de forma correcta los elementos dentro de la tabla hash).

=> La iteración a través de sus elementos es menos eficiente que en List, ya que necesitará recorrer todas las entradas de la tabla de dispersión (se debe recorrer TODA la tabla hash, toda la estructura interna que tiene varios "buckets", aunque algunos estén vacíos), lo que hará que el coste esté en función tanto del número de elementos insertados en el conjunto como del número de entradas de la tabla.

HashSet: Creación y manejo.

Creación:

```
HashSet <String> hs = new HashSet <>();
```

Al llamar al constructor, internamente el método lo que hace es crear el HashMap

Agregar elementos:

```
hs.add ("Lionel");
```

```
hs.add ("Dibu");
```

Si el objeto a agregar YA estaba agregado, no lo agrega y add retorna false.
Si el objeto a agregar NO estaba agregado, lo agrega y retorna true.
En consecuencia, es recomendable evaluar el retorno de add para ver si se agregó o no.

Ejemplos de manejo de HashSet

```
// Inicializar un HashSet
Set<String> miHashSet = new HashSet<>() ;

// Agregar elementos
miHashSet.add ("Uno") ;
miHashSet.add ("Dos") ;
miHashSet.add ("Tres") ;

// Ver el tamaño del HashSet
System.out.println ("Tamaño del HashSet: " + miHashSet.size());

// Eliminar un elemento
miHashSet.remove ( "Dos" ) ;

// Verificar si contiene un elemento
if (miHashSet.contains ("Uno") ) {
    System.out.println("El HashSet contiene el elemento 'Uno'") ;
}

// Recorrer el HashSet
Iterator<String> iterator = miHashSet.iterator () ;
while (iterator.hasNext () ) {
    String elemento = iterator.next();
    System.out.println(elemento);
}
```

LINKEDHASHSET

ES SIMILAR A HASHSET, PERO

LA TABLA HASH DE DISPERSIÓN

SE MANEJA CON UNA LISTA DOBLEMENTE ENLAZADA,

LO QUE HACE QUE

LOS ELEMENTOS CONSERVEN EL ORDEN DE INSERCIÓN

LinkedHashSet: “Costo” de las operaciones.

=> Los elementos que se inserten tendrán enlaces entre ellos. Por lo tanto, las operaciones básicas seguirán teniendo coste constante, con un ligero costo adicional por la carga extra de tener que gestionar los enlaces.

=> Sin embargo, **habrá una mejora en la iteración**, ya que al establecerse enlaces entre los elementos no tendremos que recorrer TODAS las entradas de la tabla, sólo se recorren las que tienen elementos, por lo tanto el coste sólo estará en función del número real de elementos insertados.

=> A diferencia del HashSet, acá, al haber enlaces entre los elementos, estos enlaces definirán el orden en el que se insertaron en el conjunto, por lo que **EL ORDEN DE ITERACIÓN SERÁ EL MISMO ORDEN EN EL QUE SE INSERTARON.**

Ejemplo de manejo de LinkedHashSet

```
// Creación de un LinkedHashSet
    LinkedHashSet<String> capitales = new LinkedHashSet<>();

// Añadir elementos
    capitales.add ("Madrid");
    capitales.add ("Londres");
    capitales.add ("Nueva York");

// Eliminar un elemento
    capitales.remove (o: "Londres");

// Imprimir el LinkedHashSet manteniendo el orden de inserción
    for (String capital: capitales) {
        System.out.println(capital);
    }
```

TREES



=> Almacena los elementos **UTILIZANDO UN ARBOL Y ORDENÁNDOLOS EN FUNCIÓN DE SUS VALORES.**

=> Los elementos a almacenar que pertenezcan a clases propias del sistema (String, etc) se ordenan de acuerdo a su orden natural, y a las clases propias se les deberá implementar la interfaz Comparable para que puedan ordenarse.

=> Al ser ordenado y utilizar un árbol rojo-negro (un árbol binario de búsqueda balanceado), es **más lento que HashSet para** las operaciones de **inserción o eliminación**, pero es **más rápido en** las operaciones de **búsqueda**.

TreeSet: Principales Métodos

=> **object higher(object o):** Devuelve el elemento menor de la colección, pero que sea mayor que el elemento dado. Devuelve NULL si no existe el elemento dado. EJ : `ts.higher(new Integer(5)); //6 {1-10}`

=> **object lower(object o):** Devuelve el elemento mayor de la colección, pero que sea menor que el elemento dado. Devuelve NULL si no existe el elemento dado. EJ: `ts.lower(new Integer (5)); // 4 {1-10}`

=> **object pollFirst():** Elimina el primer elemento de la colección (el primero en orden, el más “chico”). Devuelve NULL si está vacía.

=> **object pollLast():** Elimina el último elemento de la colección (el último en orden, el más “grande”) Devuelve NULL si está vacía.

TreeSet:

Ejemplo de implementación

```
// Creacion de un TreeSet
    TreeSet<String> frutas = new TreeSet<>();

// Añadir elementos
    frutas.add ("Mango") ;
    frutas.add ("Banana") ;
    frutas.add ("Manzana" );

// Acceder al primer y último elemento
    String primeraFruta = frutas.first();
    String ultimaFruta = frutas.last();

// Imprimir el TreeSet en orden natural
    for (String fruta: frutas) {
        System.out.println (fruta);
    }
```

RESUMEN DE COLECCIONES SET:

HashSet



- Rápida.
- No duplicados.
- No ordenación.
- No acc. aleatorio

LinkedHashSet



- Ordenación por entrada.
- Eficiente al acceder.
- No eficiente al agregar.

TreeSet



- Es ordenado.
- Poco eficiente.

RESUMEN COMPARATIVO DE LAS PRINCIPALES COLECCIONES SET Y LIST:

Implementación	Características	Ejemplo de uso
ArrayList	Acceso rápido por índice, redimensionamiento automático	Manejo de una lista de libros en una aplicación de biblioteca donde es frecuente acceder a libros por su índice.
LinkedList	Bueno para colas/pilas, inserciones y eliminaciones frecuentes	Implementación de una cola de tareas donde las tareas se añaden y se eliminan constantemente.
HashSet	No permite duplicados, no garantiza el orden	Almacenar una colección de direcciones de email para una campaña de marketing sin duplicados.
TreeSet	Elementos ordenados, no permite duplicados	Organizar una lista de estudiantes por apellido de manera ascendente.